TA10: Changepoint Detection in Stan

13/11/2026

In statistical analysis, change detection or change point detection tries to identify times when the probability distribution of a stochastic process or time series changes. In general the problem concerns both detecting whether or not a change has occurred, or whether several changes might have occurred, and identifying the times of any such changes.

Specific applications, like step detection and edge detection, may be concerned with changes in the mean, variance, correlation, or spectral density of the process. More generally change detection also includes the detection of anomalous behavior: anomaly detection.

Offline change point detection it is assumed that a sequence of length T is available and the goal is to identify whether any change point(s) occurred in the series. This is an example of post-hoc analysis and is often approached using hypothesis testing methods. By contrast, online change point detection is concerned with detecting change points in an incoming data stream.

Coal Mine Disaster dataset

We consider a dataset that contains information about coal mining disasters in the U.K. for the years 1851–1962. Such dataset comes from Maguire, Pearson & Wynn (1952) (https://www.jstor.org/stable/2332475) and has been revised and extended in Jarrett (1979) (https://doi.org/10.2307/2335266). Let us have a look at the data:

```
# Import modules
library(posterior)
library(bayesplot)
library(ggplot2)

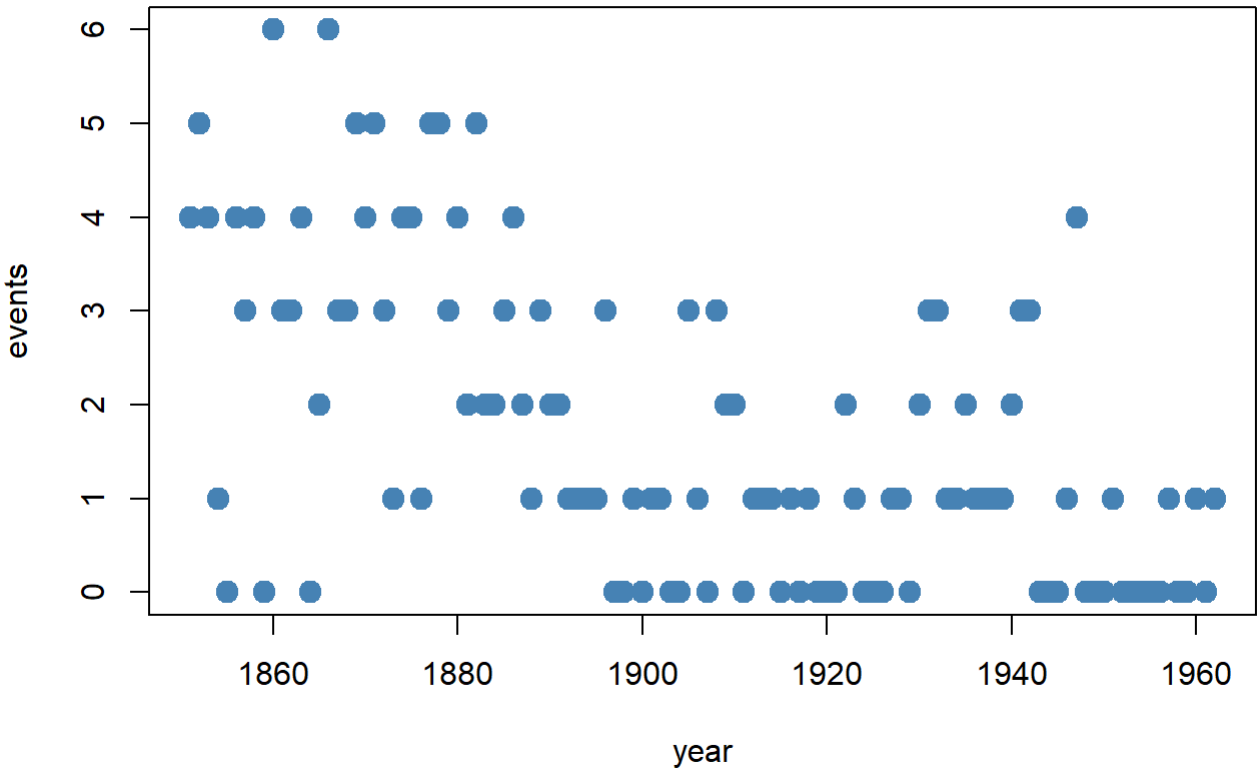
# Import functions
library(cmdstanr)

# Create ./stan folder if does not exists
if (!dir.exists("./stan")) {
    dir.create("./stan")
}
```

```
coal <- read.csv("coal.csv")
head(coal)
```

```
##   year events
## 1 1851      4
## 2 1852      5
## 3 1853      4
## 4 1854      1
## 5 1855      0
## 6 1856      4
```

```
plot(coal$year, coal$events, pch = 19, col = 'steelblue', cex = 1.5, xlab = 'year', ylab = 'events')
```



The variables are simple to interpret: for every year from 1851 to 1962 (variable `year`) the variable `event` collects the number of explosions in coal mines which resulted in 10 or more fatalities.

Fonnesbeck et al. (2013, sec. 3.1) (https://pymcmc.readthedocs.io/en/latest/tutorial.html#an-example-statistical-model) provides a Poisson model of disaster y_t in year t with two rate parameters: an early rate e and a late rate l that change at a given point in time s. The full model expressed using a latent discrete parameter s is

$$\begin{aligned} y_t \mid s, e, l &\overset{\text{iid}}{\sim} Poi\left(e\,\mathbb{I}(t < s) + l\,\mathbb{I}(t \geq s)\right) \quad t = 1, \ldots, T \\ s &\sim \mathcal{U}\left(\{1, \ldots, T\}\right) \\ e &\sim \mathcal{E}xp\left(r_e\right) \\ l &\sim \mathcal{E}xp\left(r_l\right) \end{aligned}$$

Marginalizing out the discrete parameter

To code this model in `stan`, the discrete parameter s must be marginalized out to produce a model defining the log of the probability function $\pi\left(\mathbf{y}, e, l\right)$. The full joint probability factors as:

$$\pi\left(\mathbf{y}, s, e, l\right) = \pi\left(\mathbf{y} \mid s, e, l\right) \pi(s)\pi(e)\pi(l).$$

Hence, marginalizing out s, we get the following:

$$\begin{aligned} \pi\left(\mathbf{y}, e, l\right) &= \textstyle\sum_{s=1}^{T} \pi\left(\mathbf{y} \mid s, e, l\right) \pi(s)\pi(e)\pi(l), \\ &= \left[\textstyle\sum_{s=1}^{T} \pi\left(\mathbf{y} \mid s, e, l\right) \pi(s)\right] \pi(e)\pi(l). \end{aligned}$$

Then, we can make explicit the law of $\pi\left(\mathbf{y} \mid e, l\right)$ as:

$$\sum_{s=1}^{T} \mathcal{U}\left(s \mid \{1, \ldots, T\}\right) \prod_{t=1}^{T} Poi\left(y_t \mid e\,\mathbb{I}(t < s) + l\,\mathbb{I}(t \geq s)\right)$$

Remember that `stan` works in log scale, so we need to rewrite the above equation as follows:

$$\log\left[\sum_{s=1}^{T}\exp\left\{\log\mathcal{U}\Big(s \mid \{1,\ldots,T\}\Big) + \sum_{t=1}^{T}\log Poi\Big(y_t \mid e\,\mathbb{I}(t<s) + l\,\mathbb{I}(t\geq s)\Big)\right\}\right]$$

Let's code it in `stan` !

```
coal_change_point <- "
functions {

    // Computes the marginal likelihood lpdf
    real marginal_likelihood_logpdf(array[] int obs, real early_rate, real late_rate) {

        // Deduce size of data and uniform contribution
        int T = size(obs);
        real log_unif = -log(T);

        // Compute contributions for log_sum_exp
        vector[T] lp = rep_vector(log_unif, T);
        for (s in 1:T) {
            for (t in 1:T) {
                lp[s] += poisson_lpmf(obs[t] | t < s ? early_rate : late_rate);
            }
        }

        // Return the value of the lpdf
        return log_sum_exp(lp);
    }
}

data {
    int<lower=1> T;
    array[T] int<lower=0> y;

    real<lower=0> r_e;
    real<lower=0> r_l;
}

transformed data {
    real log_unif = -log(T);
}

parameters {
    real<lower=0> e;
    real<lower=0> l;
}

model {
    // Likelihood
    target += marginal_likelihood_logpdf(y, e, l);

    // Prior
    e ~ exponential(r_e);
    l ~ exponential(r_l);
}
"

# Write stan model to file
stan_file <- "./stan/coal_change_point.stan"
writeLines(coal_change_point, stan_file)

# Compile stan model
coal_change_point_model <- cmdstan_model(stan_file)
```

```
cpc_data <- list(
    T = nrow(coal),
    y = coal$events,
    r_e = 1.0,
    r_l = 1.0
)

algo_params <- list(
    n_chains = 4,
    n_burnin = 1000,
    n_iter = 5000
)

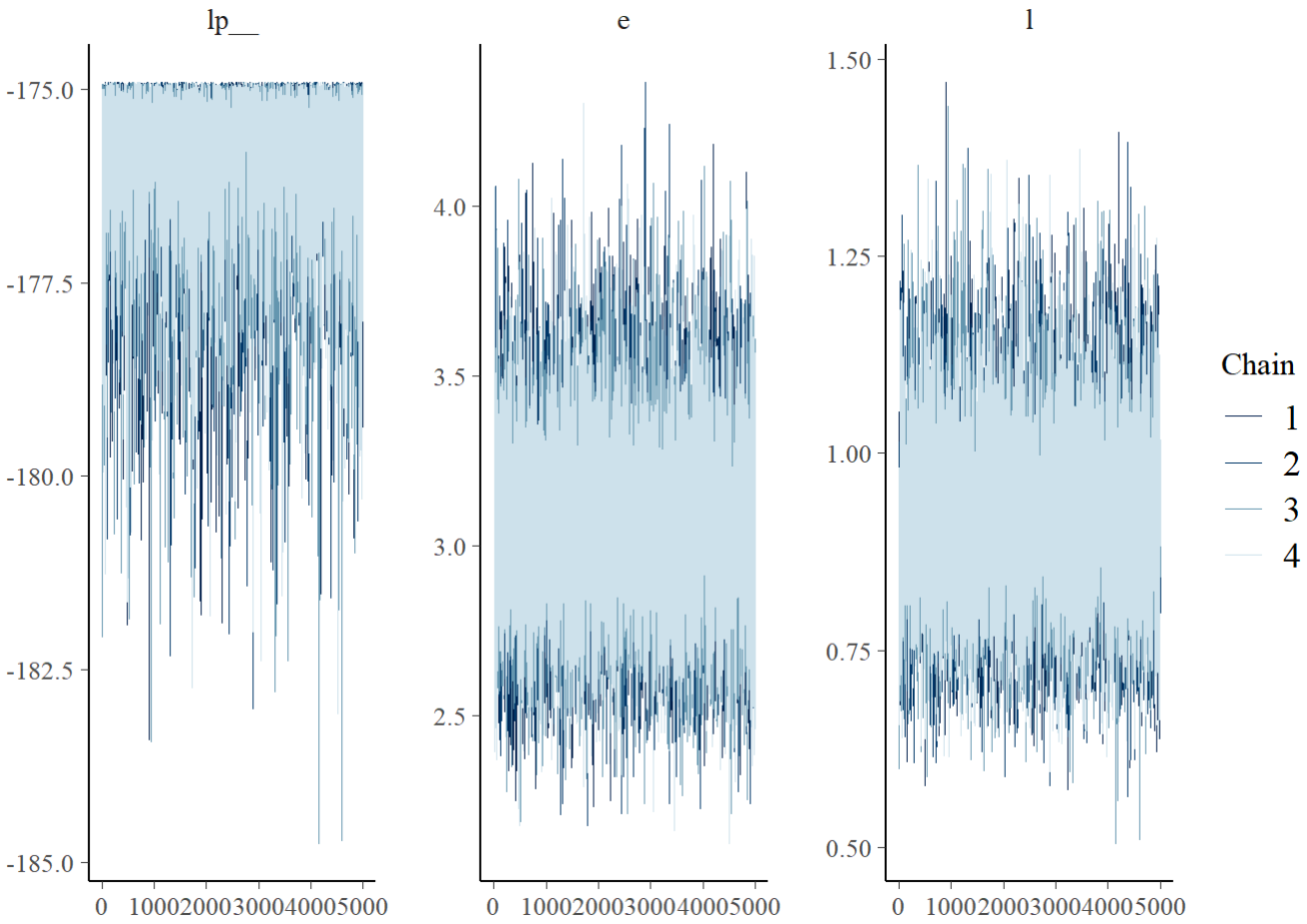
cpc_fit <- coal_change_point_model$sample(
    data = cpc_data,
    chains = algo_params$n_chains,
    parallel_chains = algo_params$n_chains,
    iter_warmup = algo_params$n_burnin,
    iter_sampling = algo_params$n_iter,
    refresh = 500,
    seed = 123,
    show_exceptions = F
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 2 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 3 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 4 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 2 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 3 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 3 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 4 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 1 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 2 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 4 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 3 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 2 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 1 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 3 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 4 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 4 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 4 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 4 finished in 10.5 seconds.
## Chain 2 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 2 finished in 10.7 seconds.
## Chain 3 Iteration: 6000 / 6000 [100%]  (Sampling)
```

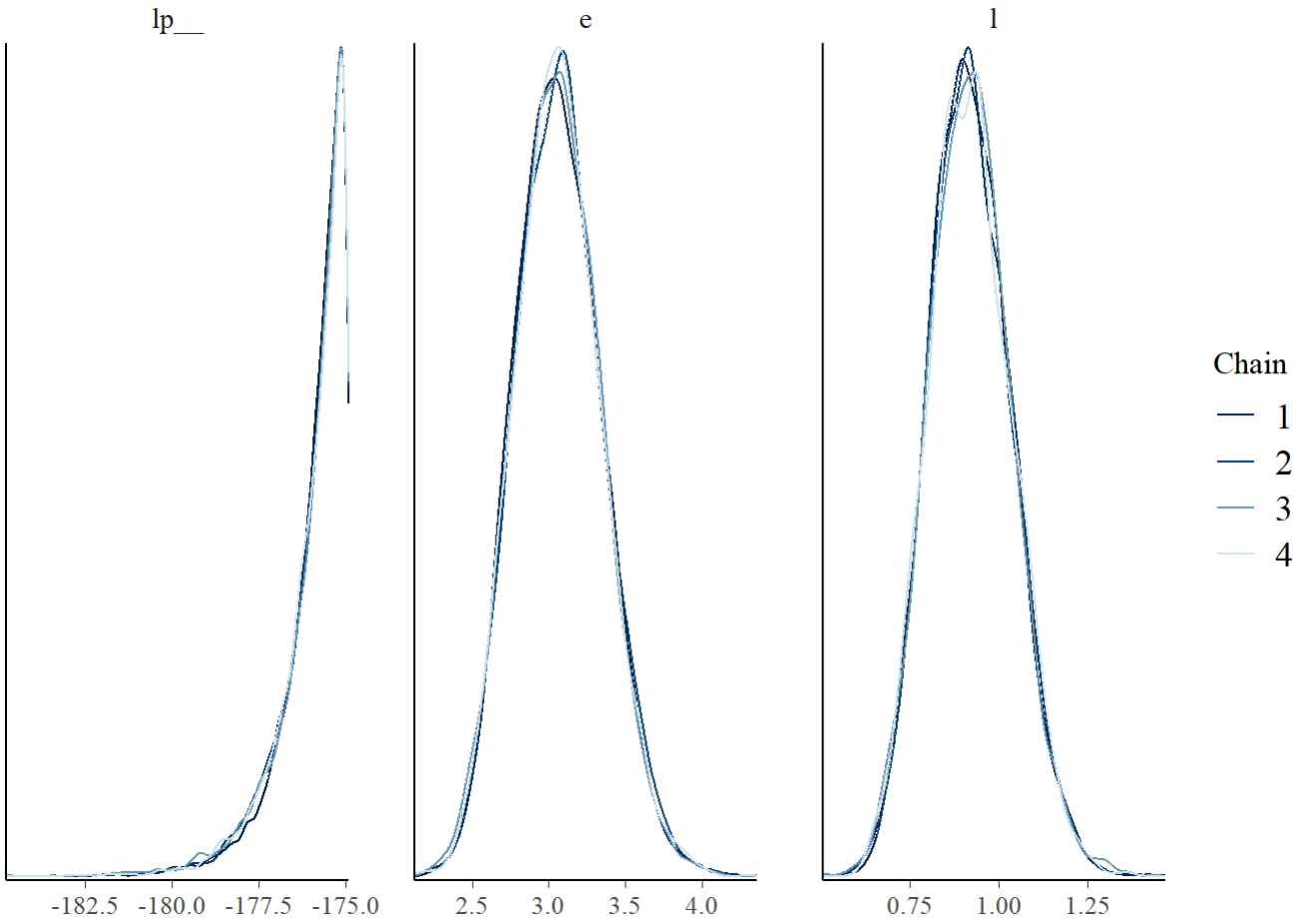
```
## Chain 3 finished in 10.7 seconds.
## Chain 1 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 1 finished in 11.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 10.8 seconds.
## Total execution time: 11.3 seconds.
```

```
posterior_draws <- posterior::as_draws_df(cpc_fit)
```

```
mcmc_trace(posterior_draws)
```




```
mcmc_dens_overlay(posterior_draws)
```



Posterior distribution of the discrete change point

Since we have marginalized the likelihood w.r.t the s variable, at this stage we do not have MCMC draws for $\pi\left(s \mid \mathbf{y}\right)$. We need to retrieve draws from its full conditional via the `generated_quantities` blocks. Thus, for $i = 1, \ldots, T$:

$$
\begin{aligned}
s = i \mid rest &\propto \pi\left(\mathbf{y} \mid s = i, e, l\right) \pi(s = i), \\
&\propto \frac{1}{T} \prod_{t=1}^{T} Poi\Big(y_t \mid e\, \mathbb{I}(t < i) + l\, \mathbb{I}(t \geq i)\Big).
\end{aligned}
$$

Thus it is a categorical random variable with probabilities proportional to the quantities above.

Let's code it in `stan` !

```
coal_change_point_v2 <- "
functions {

    // Computes vector of lp
    vector compute_logp(array[] int obs, real early_rate, real late_rate){

        // Deduce size of data and uniform contribution
        int T = size(obs);
        real log_unif = -log(T);

        // Compute contributions vector and return
        vector[T] lp = rep_vector(log_unif, T);
        for (s in 1:T) {
            for (t in 1:T) {
                lp[s] += poisson_lpmf(obs[t] | t < s ? early_rate : late_rate);
            }
        }
        return lp;

    }

    // Computes the vector of probabilites for the posterior distribution of s
    vector compute_probs(vector lp) {
        return softmax(lp);
    }

    // Computes the marginal likelihood lpdf
    real marginal_likelihood_logpdf(vector lp){
        return log_sum_exp(lp);
    }
}

data {
    int<lower=1> T;
    array[T] int<lower=0> y;

    real<lower=0> r_e;
    real<lower=0> r_l;
}

transformed data {
    real log_unif = -log(T);
}

parameters {
    real<lower=0> e;
    real<lower=0> l;
}

model {
    // Likelihood
    target += marginal_likelihood_logpdf(compute_logp(y, e, l));

    // Prior
    e ~ exponential(r_e);
    l ~ exponential(r_l);
}

generated quantities {
```

```
    int<lower=1, upper=T> s = categorical_rng(compute_probs(compute_logp(y,e,l)));
}
"

# Write stan model to file
stan_file <- "./stan/coal_change_point_v2.stan"
writeLines(coal_change_point_v2, stan_file)

# Compile stan model
coal_change_point_v2_model <- cmdstan_model(stan_file)
```

```
# Input data
cpc_data <- list(
    T = nrow(coal),
    y = coal$events,
    r_e = 1.0,
    r_l = 1.0
)

# Algorithm parameters
algo_params <- list(
    n_chains = 4,
    n_burnin = 1000,
    n_iter = 5000
)

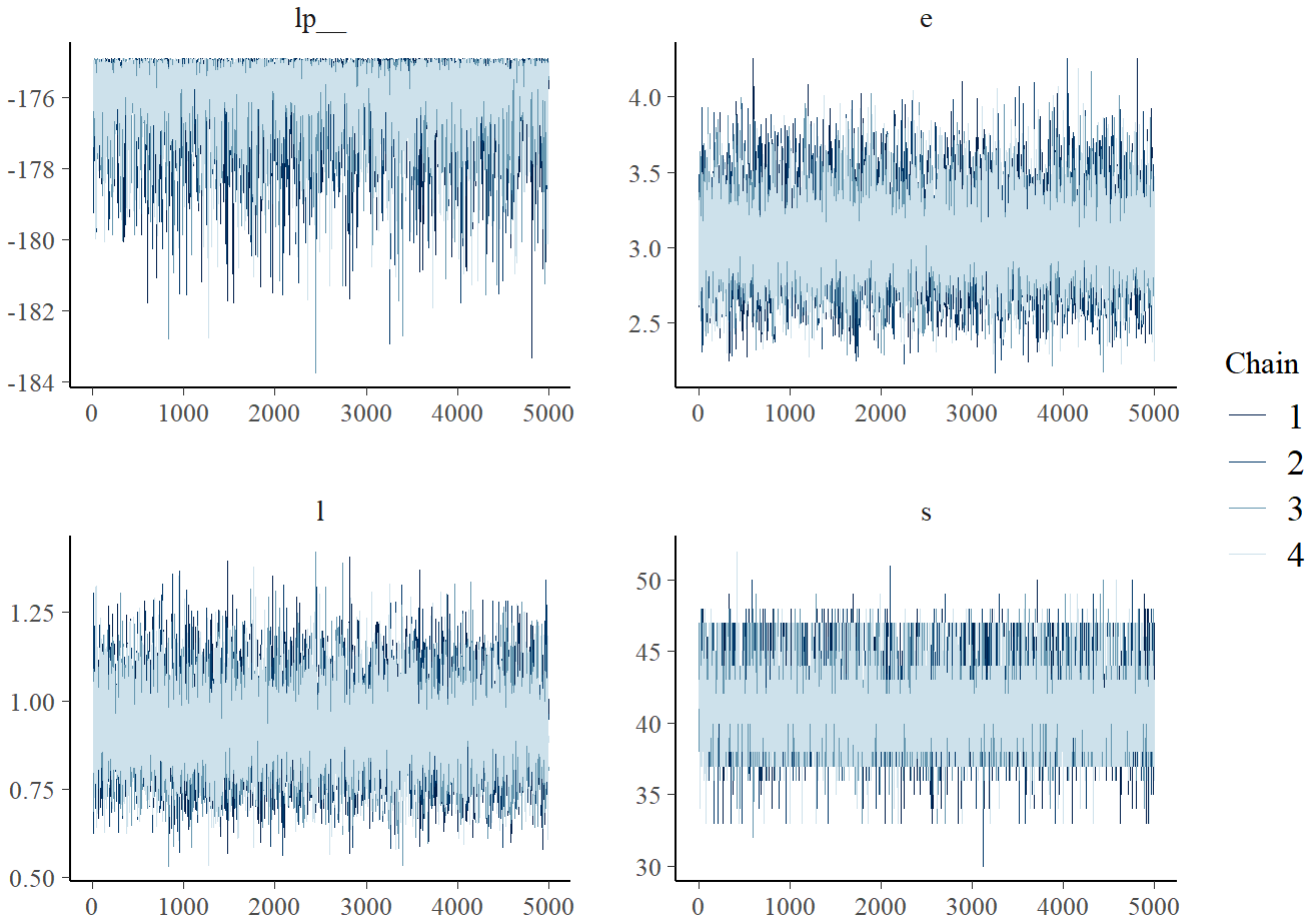
# Sample
cpc_fit <- coal_change_point_v2_model$sample(
    data = cpc_data,
    chains = algo_params$n_chains,
    parallel_chains = algo_params$n_chains,
    iter_warmup = algo_params$n_burnin,
    iter_sampling = algo_params$n_iter,
    refresh = 500,
    seed = 345,
    show_exceptions = F
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 6000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 2 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 3 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 4 Iteration:  500 / 6000 [  8%]  (Warmup)
## Chain 1 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 3 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 2 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 3 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 4 Iteration: 1000 / 6000 [ 16%]  (Warmup)
## Chain 4 Iteration: 1001 / 6000 [ 16%]  (Sampling)
## Chain 1 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 2 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 3 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 4 Iteration: 1500 / 6000 [ 25%]  (Sampling)
## Chain 2 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 3 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 1 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 4 Iteration: 2000 / 6000 [ 33%]  (Sampling)
## Chain 2 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 3 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 1 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 4 Iteration: 2500 / 6000 [ 41%]  (Sampling)
## Chain 2 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 1 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 3 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 4 Iteration: 3000 / 6000 [ 50%]  (Sampling)
## Chain 2 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 1 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 3 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 4 Iteration: 3500 / 6000 [ 58%]  (Sampling)
## Chain 2 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 1 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 3 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 4 Iteration: 4000 / 6000 [ 66%]  (Sampling)
## Chain 2 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 1 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 3 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 2 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 4 Iteration: 4500 / 6000 [ 75%]  (Sampling)
## Chain 1 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 3 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 2 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 4 Iteration: 5000 / 6000 [ 83%]  (Sampling)
## Chain 1 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 3 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 2 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 2 finished in 27.9 seconds.
## Chain 4 Iteration: 5500 / 6000 [ 91%]  (Sampling)
## Chain 1 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 1 finished in 28.6 seconds.
## Chain 3 Iteration: 6000 / 6000 [100%]  (Sampling)
```

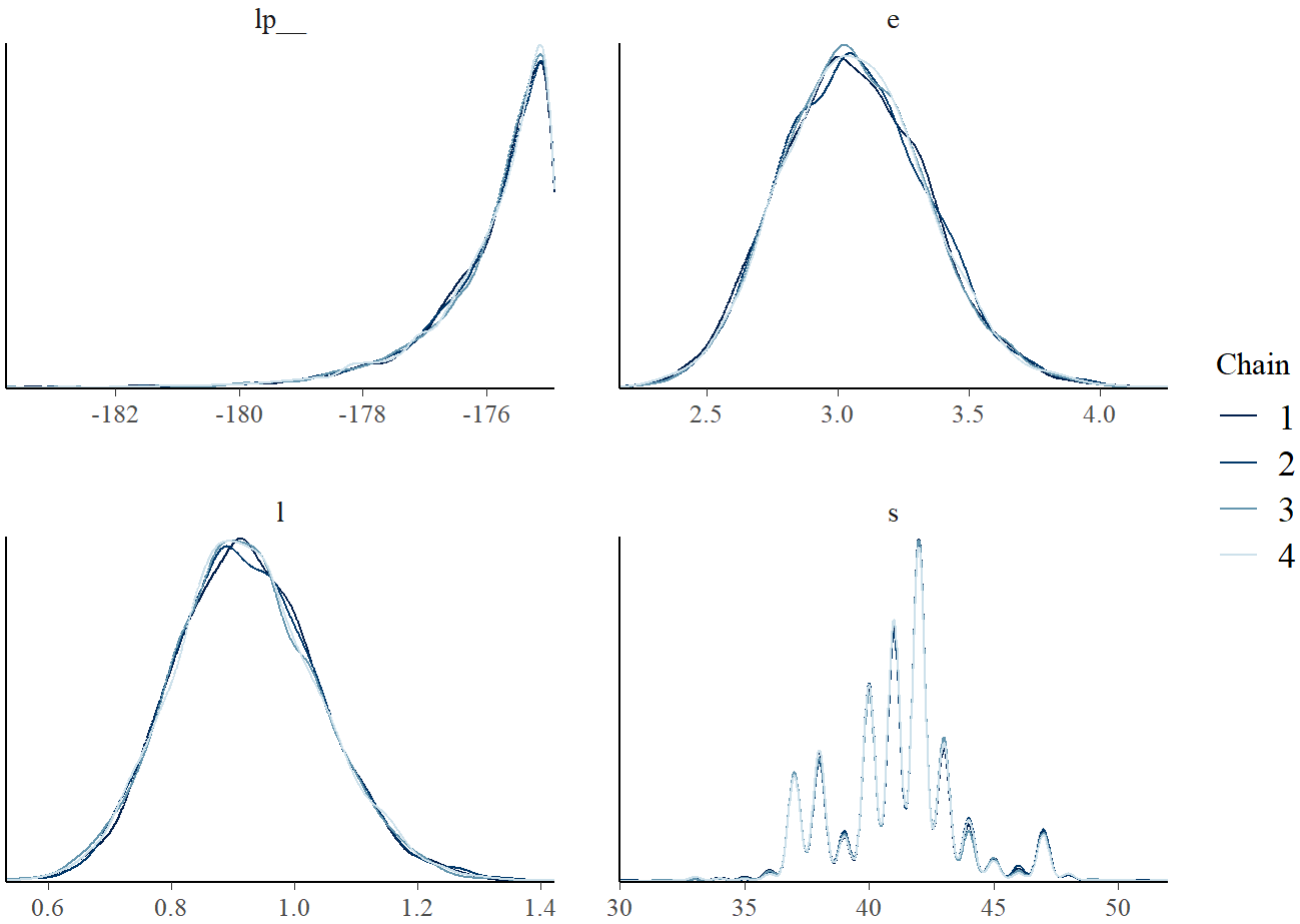
```
## Chain 3 finished in 29.2 seconds.
## Chain 4 Iteration: 6000 / 6000 [100%]  (Sampling)
## Chain 4 finished in 30.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 29.0 seconds.
## Total execution time: 30.6 seconds.
```

```
posterior_cpc_fit <- posterior::as_draws_df(cpc_fit)
```

```
mcmc_trace(posterior_cpc_fit)
```



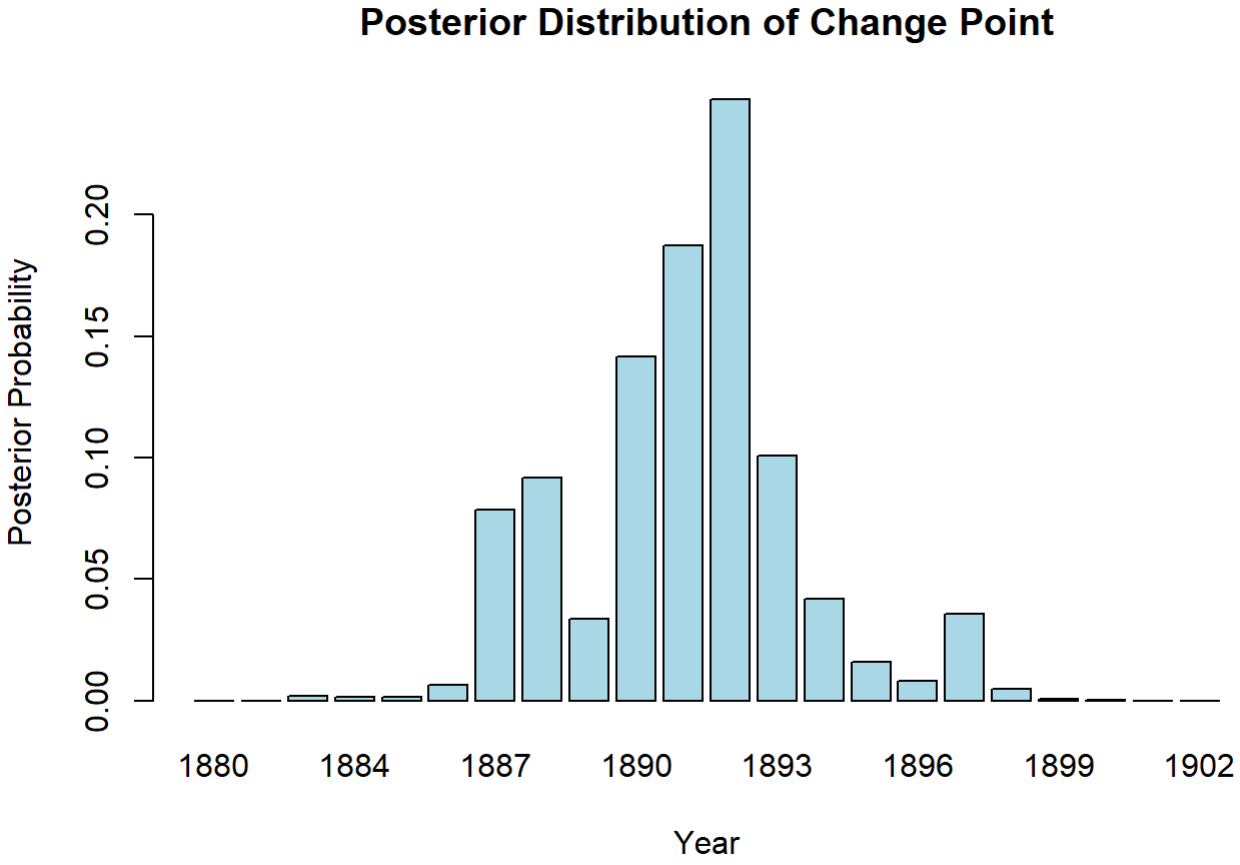

```
mcmc_dens_overlay(posterior_cpc_fit)
```




```
# Posterior distribution of s
s_chain <- as.vector(cpc_fit$draws("s"))

# Compute posterior frequencies
s_table <- table(s_chain) / length(s_chain)
s_indices <- as.integer(names(s_table))

# Posterior barplot for s
barplot(
    as.numeric(s_table),
    names.arg = coal$year[s_indices],
    col = 'lightblue',
    xlab = "Year",
    ylab = "Posterior Probability",
    main = "Posterior Distribution of Change Point"
)
```


Posterior Distribution of Change Point
Posterior Probability
0.00
0.05
0.10
0.15
0.20
1880
1884
1887
1890
1893
1896
1899
1902
Year